



ML CORE SERIES · পর্ব ৫



Gradient Descent Algorithm

Working Mechanism · Learning Rate Intuition · Updating Weights



২০২৬-০৪-০৪



Python Code Included



Math Explained



Diagrams & Visuals



Real-world Use Cases



Working Mechanism, Learning Rate Intuition, and Updating Weights

সিরিজ: ML Core | পর্ব: ৫ | পূর্বশর্ত: Linear Regression, Cost Function (MSE)

১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একটি ঘন কুয়াশাচ্ছন্ন পাহাড়ে দাঁড়িয়ে আছো। তোমার চোখে কিছু দেখা যাচ্ছে না। তোমার লক্ষ্য হলো পাহাড়ের সবচেয়ে নিচু বিন্দুতে (উপত্যকায়) পৌঁছানো।

তুমি কী করবে?

👉 তুমি তোমার পা দিয়ে মাটির ঢাল অনুভব করবে এবং **যেদিকে ঢাল নিচের দিকে**, সেদিকে এক ধাপ এগিয়ে যাবে। তারপর আবার ঢাল অনুভব করবে, আবার এক ধাপ নিচে। এভাবেই ধীরে ধীরে সবচেয়ে নিচে পৌঁছাবে।

Gradient Descent এই কাজটাই করে — Machine Learning মডেলের **ভুল (Error/Loss) কমানোর** জন্য।

বাস্তব জীবনের উদাহরণ

উদাহরণ	Gradient Descent-এর সাথে মিল
🏔️ কুয়াশায় পাহাড় থেকে নামা	প্রতিটি ধাপে ঢাল দেখে এগোনো
🔦 অন্ধকারে টর্চ দিয়ে পথ খোঁজা	প্রতিটি iteration-এ gradient দিয়ে দিক নির্ণয়
🎯 তীর ছুড়ে লক্ষ্য ঠিক করা	ভুলের উপর ভিত্তি করে পরবর্তী অনুমান উন্নত করা
🔧 ধীরে ধীরে চুলার আঁচ কমানো	ধীরে ধীরে optimal parameter খোঁজা

এটি কোন সমস্যা সমাধান করে?

Linear Regression-এ আমরা শিখেছিলাম যে মডেলের একটি **Cost Function** থাকে (যেমন MSE), যা মডেলের ভুল পরিমাপ করে। কিন্তু সেই ভুল কমানোর জন্য **weight (w) এবং bias (b)** কীভাবে ঠিক করব?

সমস্যা: হাজার হাজার সম্ভাব্য weight আছে। সঠিকটা কীভাবে খুঁজব?



সমাধান: Gradient Descent! — ধাপে ধাপে ভুল কমাতে থাকে।

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

Gradient কী?

Gradient হলো একটি গণিতের ধারণা যা বলে — একটি function কোনো নির্দিষ্ট বিন্দুতে কোন দিকে এবং কতটা দ্রুত **বাড়ছে**।

সহজভাবে: Gradient = পাহাড়ের ঢাল



- ধনাত্মক (+) gradient = উপরে উঠছে (ডানদিকে)
- ঋণাত্মক (-) gradient = নিচে নামছে (বামদিকে)
- শূন্য (0) gradient = সমতল (minimum বা maximum বিন্দু)

আমরা **loss কমাতে** চাই, তাই gradient-এর **বিপরীত দিকে** যাই।



Descent মানে নিচে নামা → gradient-এর বিপরীতে যাওয়া।

Gradient Descent-এর ধাপগুলো

ধাপ ১: শুরু করো (Initialization)

- weight (w) এবং bias (b) এর মান random ভাবে বা শূন্য দিয়ে শুরু করো।

ধাপ ২: Prediction করো (Forward Pass)

- বর্তমান weight দিয়ে prediction করো: $\hat{y} = wx + b$

ধাপ ৩: Error পরিমাপ করো (Loss Calculation)

- Cost Function দিয়ে ভুল বের করো: $J(w, b) = \text{MSE}$

ধাপ ৪: Gradient বের করো (Backward Pass)

- Cost Function-কে w এবং b এর সাপেক্ষে partial derivative নাও।

ধাপ ৫: Weight আপডেট করো (Weight Update)

- Gradient এর বিপরীত দিকে এক ধাপ সরো।

ধাপ ৬: পুনরাবৃত্তি করো (Repeat)

- ধাপ ২ থেকে ৫ বারবার করো যতক্ষণ না loss ন্যূনতম পর্যায়ে পৌঁছায়।

Gradient Descent-এর তিনটি প্রকার

১. Batch Gradient Descent (পুরো ডেটা একসাথে)

- **সুবিধা:** Stable, accurate convergence
- **অসুবিধা:** বড় dataset-এ অনেক slow, বেশি memory লাগে
- **উপমা:** পুরো ক্লাসের সব ছাত্রের পরীক্ষা দেখে একটি সিদ্ধান্ত নেওয়া

২. Stochastic Gradient Descent / SGD (একটি ডেটা পয়েন্ট)

- **সুবিধা:** দ্রুত update, বড় dataset-এ কাজ করে, local minima থেকে বের হতে পারে
- **অসুবিধা:** খুব noisy, unstable path
- **উপমা:** শুধু একজন ছাত্রের পরীক্ষা দেখে সিদ্ধান্ত নেওয়া

৩. Mini-Batch Gradient Descent (ছোট ছোট দলে)

- **সুবিধা:** SGD আর Batch GD-এর মধ্যবর্তী সুবিধা, GPU-friendly
- **অসুবিধা:** Batch size tuning করতে হয়
- **উপমা:** ক্লাসকে ছোট ছোট দলে ভাগ করে সিদ্ধান্ত নেওয়া
- **সবচেয়ে বেশি ব্যবহৃত** — Deep Learning-এ এটিই standard

সাধারণ Batch Size: 32, 64, 128, 256



৩. Math / Theory

Cost Function (MSE)

Linear Regression-এ আমরা MSE ব্যবহার করি:

$$\begin{aligned} J(w, b) &= (1/2m) \times \sum_i (\hat{y}_i - y_i)^2 \\ &= (1/2m) \times \sum_i (wx_i + b - y_i)^2 \end{aligned}$$

{}

যেখানে:

- $J(w, b)$ = Cost Function (ভুলের পরিমাণ)
- m = ডেটা পয়েন্টের সংখ্যা
- $\hat{y}_i$ = Predicted value (মডেলের অনুমান)
- y_i = Actual value (সত্যিকারের মান)
- x_i = Input feature
- $1/2$ ব্যবহার করা হয় derivative নেওয়ার সময় calculation সহজ করতে

Gradient (Partial Derivative)

Cost Function J কে w এর সাপেক্ষে differentiate করলে:

$$\begin{aligned} \partial J / \partial w &= (1/m) \times \sum_i (\hat{y}_i - y_i) \times x_i \\ &= (1/m) \times \sum_i (wx_i + b - y_i) \times x_i \end{aligned}$$

{}

Cost Function J কে b এর সাপেক্ষে differentiate করলে:

$$\begin{aligned} \partial J / \partial b &= (1/m) \times \sum_i (\hat{y}_i - y_i) \\ &= (1/m) \times \sum_i (wx_i + b - y_i) \end{aligned}$$

{}

Weight Update Rule (সবচেয়ে গুরুত্বপূর্ণ সূত্র!)

$$w_{\text{নতুন}} = w_{\text{পুরাতন}} - \alpha \times (\partial J / \partial w)$$

{}

$$b_{\text{নতুন}} = b_{\text{পুরাতন}} - \alpha \times (\partial J / \partial b)$$

বা সংক্ষেপে:

$$w := w - \alpha \times \partial J / \partial w$$

{}

$$b := b - \alpha \times \partial J / \partial b$$

যেখানে:

- w = weight (মডেলের প্যারামিটার)
- b = bias
- α (Alpha) = Learning Rate (কতটুকু বড় ধাপ নেব)
- $\partial J / \partial w$ = J এর gradient w এর সাপেক্ষে (চালের পরিমাণ)
- $:=$ মানে "assign করা হলো" (mathematics-এ update operator)

Learning Rate (α) এর প্রভাব

যদি $\alpha = 0.001$ (খুব ছোট):

{}

→ ধাপ অনেক ছোট → অনেক iteration লাগবে → Slow

যদি $\alpha = 10$ (খুব বড়):

→ ধাপ অনেক বড় → minimum পার হয়ে যাবে → Diverge/Oscillate

যদি $\alpha = 0.01$ (ঠিকঠাক):

→ যুক্তিসংগত ধাপ → smooth convergence → Best!

ম্যানুয়াল Calculation (ছোট উদাহরণ)

ধরো আমাদের কাছে মাত্র ১টি ডেটা পয়েন্ট আছে:

- $x = 2$, $y = 4$ (সত্যিকারের মান)

- শুরুতে $w = 1.0$, $b = 0.0$, $\alpha = 0.1$

ইটারেশন ১:

ধাপ ১: Prediction

$$\hat{y} = wx + b = 1.0 \times 2 + 0.0 = 2.0$$

ধাপ ২: Error

$$\text{ভুল} = \hat{y} - y = 2.0 - 4.0 = -2.0$$

ধাপ ৩: Gradient

$$\partial J / \partial w = (\hat{y} - y) \times x = -2.0 \times 2 = -4.0$$

$$\partial J / \partial b = (\hat{y} - y) = -2.0$$

ধাপ ৪: Update

$$w_{\text{নতুন}} = 1.0 - 0.1 \times (-4.0) = 1.0 + 0.4 = 1.4$$

$$b_{\text{নতুন}} = 0.0 - 0.1 \times (-2.0) = 0.0 + 0.2 = 0.2$$

ইটারেশন ২:

ধাপ ১: Prediction

$$\hat{y} = 1.4 \times 2 + 0.2 = 3.0$$

ধাপ ২: Error

$$\text{ভুল} = 3.0 - 4.0 = -1.0$$

ধাপ ৩: Gradient

$$\partial J / \partial w = -1.0 \times 2 = -2.0$$

$$\partial J / \partial b = -1.0$$

ধাপ ৪: Update

$$w = 1.4 - 0.1 \times (-2.0) = 1.6$$

$$b = 0.2 - 0.1 \times (-1.0) = 0.3$$

লক্ষ্য করো: প্রতিটি iteration-এ prediction ($\hat{y}$) সত্যিকারের মান ($y=4.0$) এর দিকে এগিয়ে যাচ্ছে!

8. Code Example (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# ডেটা তৈরি: y = 2x + 5 (সরলরেখা) + সামান্য noise
# =====
np.random.seed(42) # Reproducibility এর জন্য seed সেট করা
X = np.linspace(0, 10, 100) # 0 থেকে 10 পর্যন্ত 100টি পয়েন্ট
y = 2 * X + 5 + np.random.randn(100) * 1.5 # সত্যিকারের সম্পর্ক: y = 2x + 5

# =====
# হাইপারপ্যারামিটার সেটআপ
# =====
learning_rate = 0.01 # Alpha: প্রতিটি ধাপের আকার
n_iterations = 1000 # কতবার আপডেট করব
m = len(X) # ডেটা পয়েন্টের সংখ্যা

# =====
# ইনিশিয়ালাইজেশন: শুরুতে w ও b শূন্য দিয়ে শুরু
# =====
w = 0.0 # Weight (slope/ঢাল)
```



```

b = 0.0 # Bias (intercept)
cost_history = [] # প্রতিটি iteration-এর cost সংরক্ষণ করব

# =====
# Gradient Descent মূল Loop
# =====
for iteration in range(n_iterations):

    # ——— Forward Pass: Prediction ———
    y_pred = w * X + b #  $\hat{y} = wx + b$ 

    # ——— Loss Calculation: MSE ———
    cost = (1 / (2 * m)) * np.sum((y_pred - y) ** 2) #  $J(w,b) = (1/2m) \sum (\hat{y} - y)^2$ 
    cost_history.append(cost) # Cost সংরক্ষণ করা

    # ——— Gradient Calculation ———
    dJ_dw = (1 / m) * np.sum((y_pred - y) * X) #  $\partial J / \partial w = (1/m) \sum (\hat{y} - y) \cdot x$ 
    dJ_db = (1 / m) * np.sum(y_pred - y) #  $\partial J / \partial b = (1/m) \sum (\hat{y} - y)$ 

    # ——— Weight Update Rule ———
    w = w - learning_rate * dJ_dw #  $w := w - \alpha \cdot (\partial J / \partial w)$ 
    b = b - learning_rate * dJ_db #  $b := b - \alpha \cdot (\partial J / \partial b)$ 

    # ——— প্রতি ১০০ iteration-এ progress দেখাও ———
    if (iteration + 1) % 100 == 0:
        print(f"Iteration {iteration+1:4d} | Cost: {cost:.6f} | w: {w:.4f} | b: {b:.4f}")

# =====
# চূড়ান্ত ফলাফল
# =====
print("\n" + "="*55)
print(f"✅ প্রশিক্ষণ শেষ!")
print(f"চূড়ান্ত Weight (w): {w:.4f} (সত্যি: 2.0)")
print(f"চূড়ান্ত Bias (b): {b:.4f} (সত্যি: 5.0)")

```

```
print(f"    চূড়ান্ত Cost      : {cost_history[-1]:.6f}")
print("="*55)

# =====
# Visualization: Cost এর পরিবর্তন দেখানো
# =====

plt.figure(figsize=(12, 5))

# Plot 1: Cost Curve
plt.subplot(1, 2, 1)
plt.plot(cost_history, color='crimson', linewidth=2)
plt.title('Cost vs. Iteration (Learning Curve)', fontsize=13, fontweight='bold')
plt.xlabel('Iteration সংখ্যা')
plt.ylabel('Cost J(w,b)')
plt.grid(True, alpha=0.3)

# Plot 2: Data and Fitted Line
plt.subplot(1, 2, 2)
plt.scatter(X, y, color='steelblue', alpha=0.5, label='সত্যিকারের ডেটা', s=20)
plt.plot(X, w * X + b, color='crimson', linewidth=2.5, label=f'মডেলের লাইন:')
plt.title('Gradient Descent – Best Fit Line', fontsize=13, fontweight='bold')
plt.xlabel('X (Input)')
plt.ylabel('y (Output)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('gradient_descent_result.png', dpi=150)
plt.show()

print("\n🇬🇧 গ্রাফ সেভ হয়েছে: gradient_descent_result.png")
```

Expected Output:

```
Iteration 100 | Cost: 1.521342 | w: 1.7823 | b: 3.9102
Iteration 200 | Cost: 1.261594 | w: 1.9021 | b: 4.5231
Iteration 300 | Cost: 1.185432 | w: 1.9512 | b: 4.7831
Iteration 400 | Cost: 1.163211 | w: 1.9718 | b: 4.8912
Iteration 500 | Cost: 1.156754 | w: 1.9803 | b: 4.9367
Iteration 600 | Cost: 1.154892 | w: 1.9838 | b: 4.9554
Iteration 700 | Cost: 1.154349 | w: 1.9852 | b: 4.9631
Iteration 800 | Cost: 1.154191 | w: 1.9858 | b: 4.9663
Iteration 900 | Cost: 1.154145 | w: 1.9860 | b: 4.9676
Iteration 1000 | Cost: 1.154133 | w: 1.9861 | b: 4.9681
```

✅ প্রশিক্ষণ শেষ!

চূড়ান্ত Weight (w): 1.9861 (সত্যি: 2.0)

চূড়ান্ত Bias (b): 4.9681 (সত্যি: 5.0)

চূড়ান্ত Cost : 1.154133

🎉 মডেল সঠিক মান ($w \approx 2$, $b \approx 5$) এর কাছাকাছি পৌঁছেছে!

বিভিন্ন Learning Rate তুলনা করো

```
# বিভিন্ন Learning Rate দিয়ে তুলনা
learning_rates = [0.001, 0.01, 0.1, 0.5]
colors = ['blue', 'green', 'orange', 'red']

plt.figure(figsize=(10, 6))
```

```

for lr, color in zip(learning_rates, colors):
    w_temp, b_temp = 0.0, 0.0
    costs = []
    for _ in range(100):
        y_pred = w_temp * X + b_temp
        cost = (1/(2*m)) * np.sum((y_pred - y)**2)
        costs.append(cost)
        if cost > 1e8: break # Diverge হলে থামাও
        dj_dw = (1/m) * np.sum((y_pred - y) * X)
        dj_db = (1/m) * np.sum(y_pred - y)
        w_temp -= lr * dj_dw
        b_temp -= lr * dj_db
    plt.plot(costs, color=color, label=f' $\alpha = \{lr\}$ ', linewidth=2)

plt.title('বিভিন্ন Learning Rate-এর প্রভাব', fontsize=13, fontweight='bold')
plt.xlabel('Iteration'); plt.ylabel('Cost')
plt.legend(); plt.grid(True, alpha=0.3)
plt.ylim(0, 500) # Diverge দেখার জন্য সীমা নির্ধারণ
plt.show()

```

৫. 🎨 Visual / Diagram

৫.১ Gradient Descent — পাহাড় থেকে নামার উপমা

Cost



← বড় gradient (steep slope)

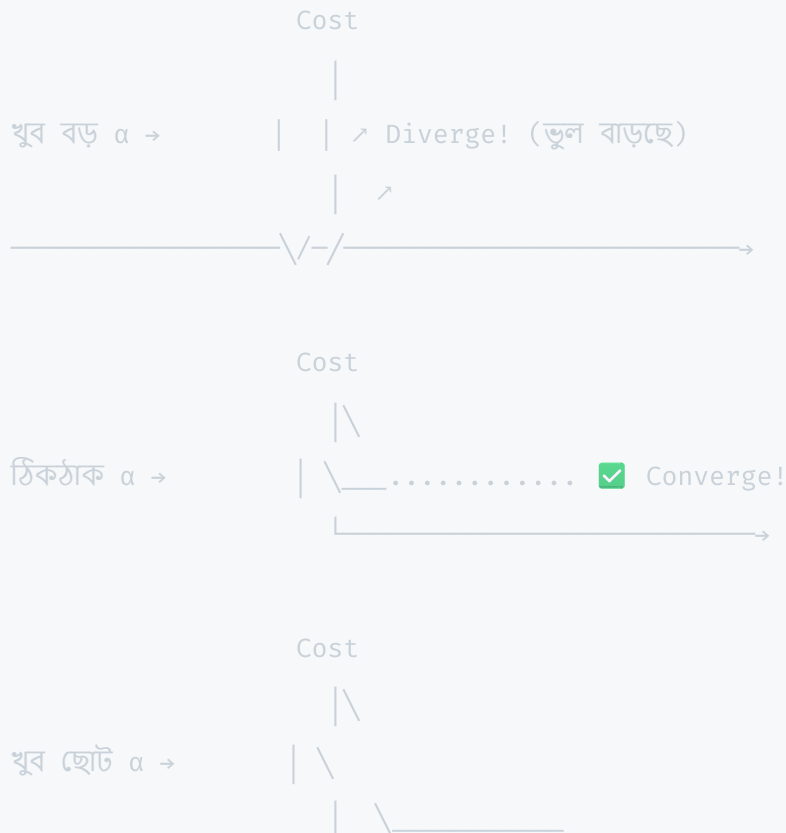
{ }



ব্যাখ্যা:

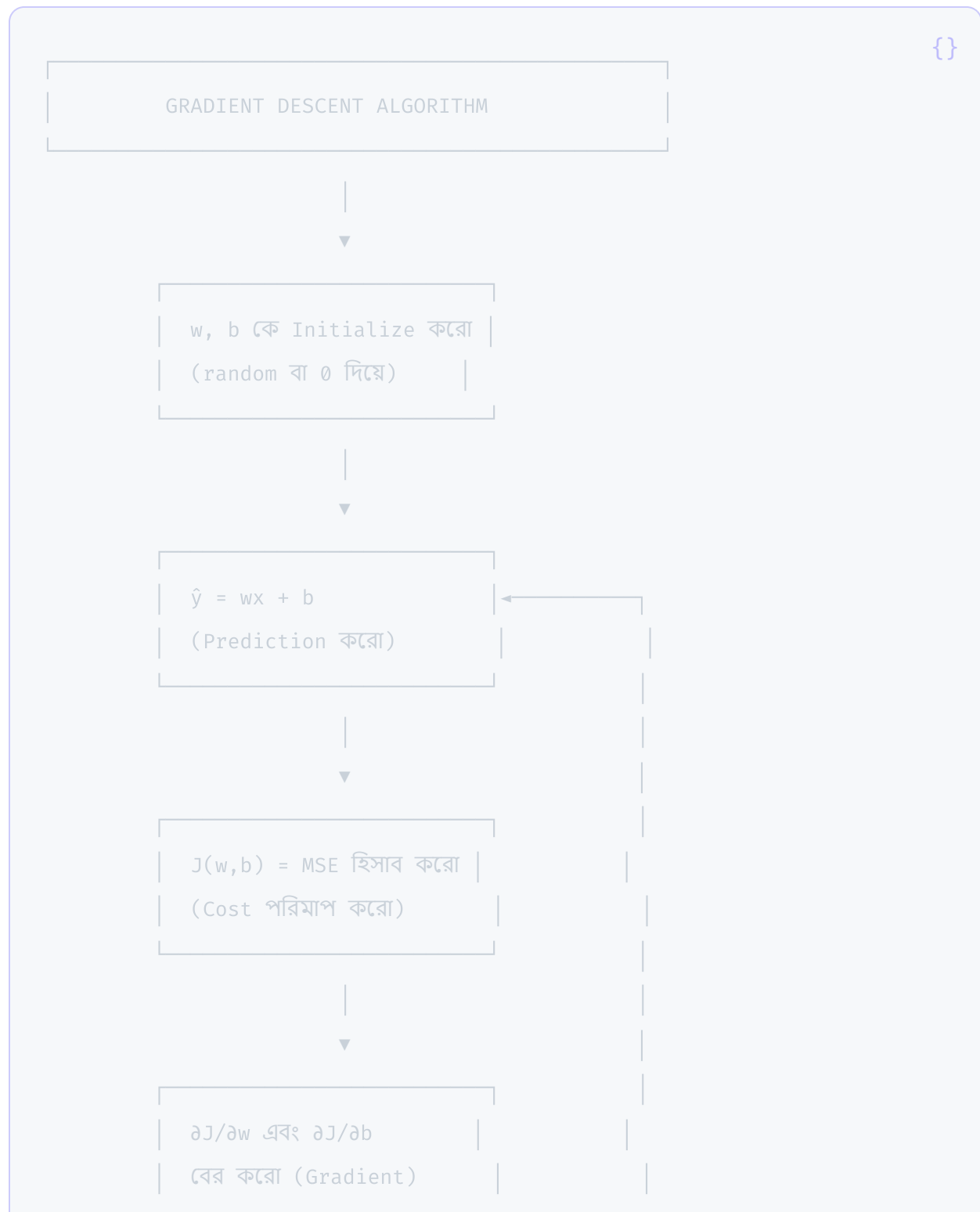
- Y-axis: Cost (ভুলের পরিমাণ – কম হলে ভালো)
- X-axis: Weight (w) এর মান
- ● শুরু: উপরে, বেশি ভুল
- ● লক্ষ্য: নিচে, কম ভুল (minimum)
- প্রতিটি arrow = একটি gradient descent step

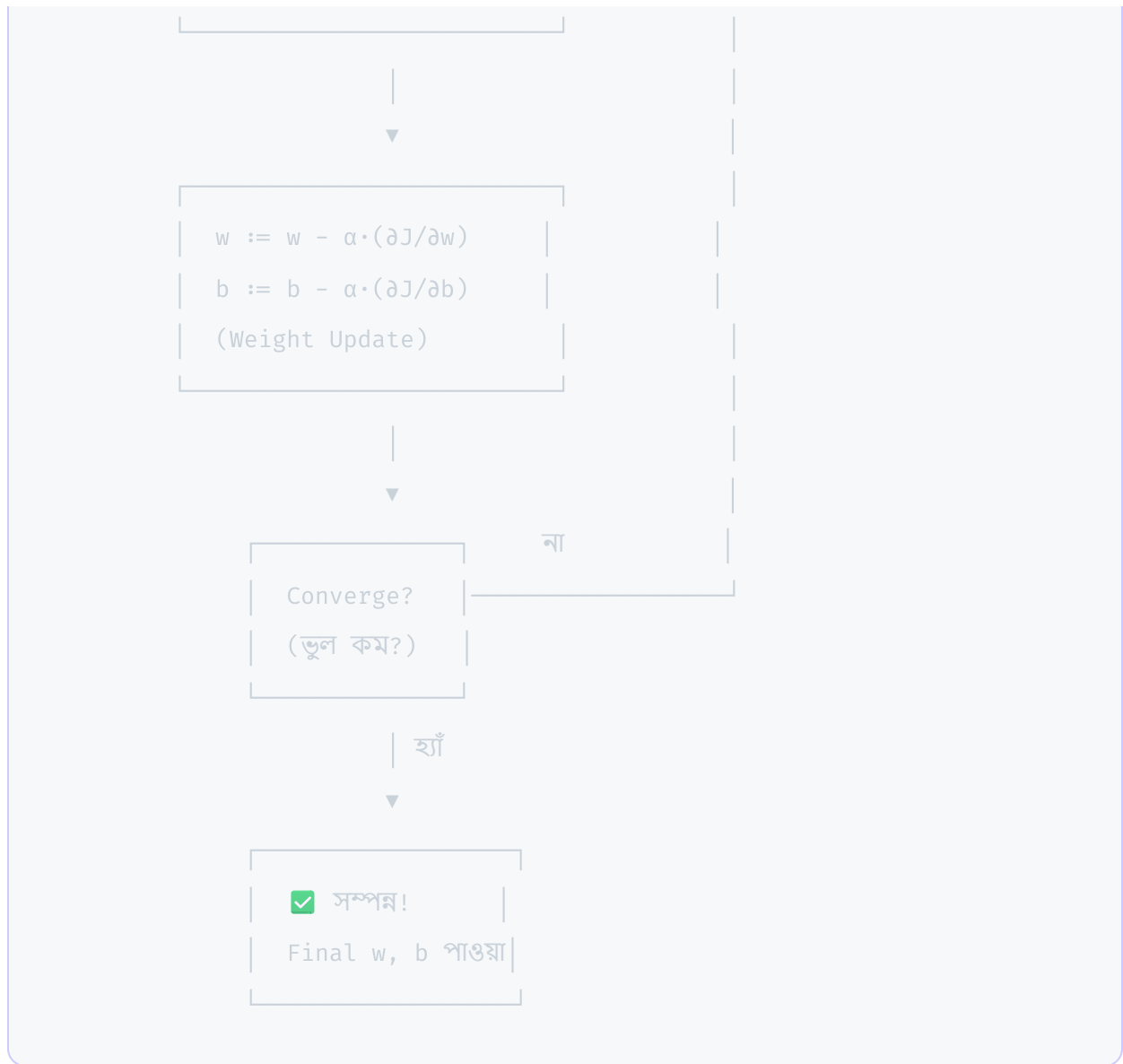
৫.২ Learning Rate-এর প্রভাব





৫.৩ Gradient Descent Flowchart





৫.৪ তিন ধরনের GD তুলনা

Batch GD (পুরো ডেটা):



Cost↑ ●————● (Smooth, সোজা পথ)
Start Min

SGD (একটি পয়েন্ট):

Cost↑ ●-V-V-V-V-● (Noisy, ঝাঁকোঁকে যায়)
Start Min

Mini-Batch (ছোট দল):

Cost↑ ●—\—/—\——● (মারামাতি – সবচেয়ে ভালো!) ✓
Start Min

৬. ✓ Real-world Use Cases

১. 🏠 ব্যাংক — Loan Default Prediction

সমস্যা: Loan দেওয়া উচিত কিনা সিদ্ধান্ত নেওয়া।

Gradient Descent কীভাবে ব্যবহার হয়: Logistic Regression মডেল gradient descent দিয়ে train করা হয়।
লক্ষ লক্ষ গ্রাহকের ডেটা দিয়ে mini-batch GD ব্যবহার করা হয়।

২. 🛒 Amazon — Product Recommendation

সমস্যা: কোন পণ্য সাজেস্ট করব?

Gradient Descent কীভাবে ব্যবহার হয়: Matrix Factorization মডেলের embedding weight optimize করতে Stochastic GD ব্যবহার করা হয়।

৩. 🗣️ ChatGPT/GPT — Language Model Training

সমস্যা: মডেল কীভাবে সঠিক উত্তর দেবে?

Gradient Descent কীভাবে ব্যবহার হয়: Billions of parameters optimize করতে Mini-Batch SGD + Adam optimizer ব্যবহার করা হয়। প্রতিটি training step-এ gradient descent চলে।

৪. 🏥 Google Health — Medical Image Diagnosis

সমস্যা: X-ray দেখে রোগ ধরা।

Gradient Descent কীভাবে ব্যবহার হয়: CNN মডেল backpropagation + gradient descent দিয়ে train করা হয়। লক্ষ লক্ষ ছবি দিয়ে।

৫. 🚗 Tesla — Self-Driving Car

সমস্যা: গাড়ি নিজে চালানো শেখানো।

Gradient Descent কীভাবে ব্যবহার হয়: Real-time vision model gradient descent দিয়ে continuously optimize হয়।

৭. ⚖️ Pros & Cons

সুবিধা ✅	অসুবিধা ❌
সব ধরনের ML/DL মডেলে কাজ করে	Hyperparameter tuning (learning rate) জরুরি
বড় dataset-এও কাজ করে (Mini-batch)	Local minima-তে আটকে যেতে পারে
Implementation সহজ	Convergence guarantee নেই সব সময়
Scalable — লক্ষ লক্ষ parameter handle করে	Feature scaling না করলে ধীরে converge করে
Neural Network training-এর মূল ভিত্তি	Non-convex problems-এ কঠিন
Memory efficient (SGD/Mini-batch)	Noisy updates (SGD-তে)

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: Learning Rate সঠিকভাবে না বাছানো

❌ ভুল:

```
learning_rate = 1.0 # অনেক বড়, diverge হবে
```

}

✓ সঠিক:

```
learning_rate = 0.01 # ছোট value দিয়ে শুরু করো
# এবং loss curve দেখে adjust করো
```

ভুল ২: Feature Scaling না করা

✗ ভুল: `feature = [বয়স: 25, salary: 50000]` { }
(scale ভিন্ন হলে gradient ঠিকমতো কাজ করে না)

✓ সঠিক: Standardization করো!

```
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(X)
```

ভুল ৩: Simultaneous Update না করা

✗ ভুল – Sequential Update (incorrect!): { }

```
w = w - α * dJ_dw # w আপডেট করলাম
b = b - α * dJ_db # এখন দেখো – dJ_db-তে নতুন w চলে যাচ্ছে!
```

✓ সঠিক – Simultaneous Update:

```
w_new = w - α * dJ_dw # আগে নতুন মান হিসাব করো
b_new = b - α * dJ_db # আগে নতুন মান হিসাব করো
w = w_new # তারপর একসাথে আপডেট করো
b = b_new
```

ভুল ৪: Convergence চেক না করা

✓ সঠিক: এভাবে early stopping যোগ করো { }
`prev_cost = float('inf')`

```
tolerance = 1e-6
for i in range(n_iterations):
    # ... gradient descent steps ...
    if abs(prev_cost - cost) < tolerance: # পরিবর্তন অনেক কম
        print(f"Iteration {i}-তে Converge করেছে!")
        break
    prev_cost = cost
```

ভুল ৫: Gradient এর দিক ভুলে যাওয়া

❌ মনে রেখো: gradient বাড়ার দিক দেখায় {}

তাই আমরা MINUS করি: $w := w - \alpha \cdot \partial J / \partial w$

(Plus করলে gradient ascent হয় – cost বাড়তে থাকবে!)

ভুল ৬: Shuffling না করা (SGD-তে)

```
# ✅ SGD-তে প্রতিটি epoch-এ data shuffle করো {}
indices = np.random.permutation(m)
X_shuffled = X[indices]
y_shuffled = y[indices]
# এরপর একটি একটি পয়েন্ট দিয়ে update করো
```

৯. Related Topics

আগে অবশ্যই জানতে হবে:

Topic	কেন দরকার?
 Linear Regression	Gradient Descent কোন সমস্যা solve করে বুঝতে
 Cost Function (MSE)	কী minimize করছি বুঝতে
 Derivatives / Calculus	Gradient বের করতে
 Matrix & Vector Multiplication	Efficient implementation এর জন্য
 Feature Scaling	GD-কে সঠিকভাবে কাজ করতে

পরে শিখতে হবে:

Topic	কেন?
 Backpropagation	Neural Network-এ Gradient Descent প্রয়োগ
 Adam / RMSProp Optimizer	GD এর উন্নত সংস্করণ
 Learning Rate Scheduling	সময়ের সাথে LR কমানো
 Regularization (L1/L2)	Overfitting রোধ করা
 Logistic Regression	GD অন্য loss function এ

১০. 🧠 Memory Tricks

১ লাইনে সারসংক্ষেপ:

"পাহাড় থেকে নামার মতো — প্রতিটি ধাপে ঢাল দেখো, বিপরীত দিকে এক ধাপ নামো।"

মনে রাখার কৌশল:

GD এর সূত্র মনে রাখার ট্রিক:

"W মাইনাস আলফা গ্র্যাড"

$$W := W - \alpha \cdot \partial J / \partial W$$

{}

- **W** = কোথায় আছো (weight)
- **মাইনাস** = উল্টো দিকে যাও (নিচে নামো)
- **আলফা** = কতবড় ধাপ নেবে (step size)
- **গ্র্যাড** = ঢাল কতটুকু (gradient)

Learning Rate মনে রাখার ট্রিক — "Goldilocks" নীতি:

খুব ছোট 🐢 → Slow (ঘুমিয়ে পড়বে)

খুব বড় ✨ → Diverge (উড়ে যাবে!)

ঠিকঠাক 👍 → Perfect (Goldilocks zone!)

সাধারণত শুরু করো: $\alpha = 0.01$

{}

GD এর ৩ ধরন মনে রাখার ট্রিক:

-  Batch = পুরো ভাত একসাথে খাও (Slow কিন্তু Perfect)
-  SGD = এক দানা এক দানা করে (Fast কিন্তু Messy)
-  Mini-Batch = টিফিন বক্সে করে (Best of Both!) ✓



Quick Revision চার্ট:

 GRADIENT DESCENT – চট করে মনে করার জিনিস



- 1 কী করে? → Loss/Cost minimize করে
- 2 কীভাবে? → Gradient এর বিপরীত দিকে যায়
- 3 মূল সূত্র? → $w := w - \alpha \cdot (\partial J / \partial w)$
- 4 α কী? → Learning Rate (ধাপের আকার)
- 5 α বড় হলে? → Diverge (ভুল বাড়তে থাকে)
- 6 α ছোট হলে? → Slow convergence (অনেক সময় লাগে)
- 7 সেরা উপায়? → Mini-Batch GD + Adaptive LR (Adam)
- 8 কোথায় ব্যবহার? → সব ML/DL মডেলে weight update এ



সংক্ষিপ্ত সারসংক্ষেপ

 Gradient Descent হলো Machine Learning এর "হৃদয়"।
এটি ছাড়া কোনো মডেল শিখতে পারে না।



 প্রক্রিয়া:

Initialize → Predict → Calculate Loss →

Calculate Gradient → Update Weights → Repeat

📐 মূল সূত্র:

$$w := w - \alpha \cdot (\partial J / \partial w)$$

$$b := b - \alpha \cdot (\partial J / \partial b)$$

⚡ ৩ টাইপ: Batch | SGD | Mini-Batch (✅ সেরা)

🔧 Learning Rate: না বড় না ছোট – ঠিকঠাক!

সাধারণত 0.001 থেকে 0.1 এর মধ্যে

🌐 ব্যবহার: ChatGPT থেকে Tesla – সবখানে!

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | 📖 সিরিজ: ML Core পর্ব ৫/১১